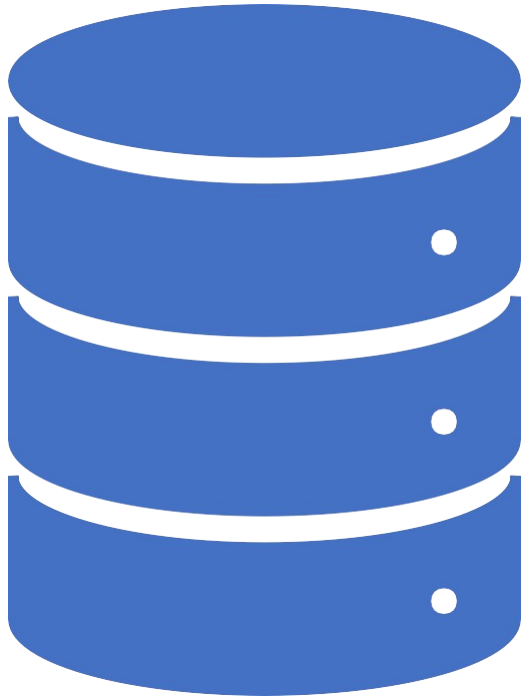




Database Systems

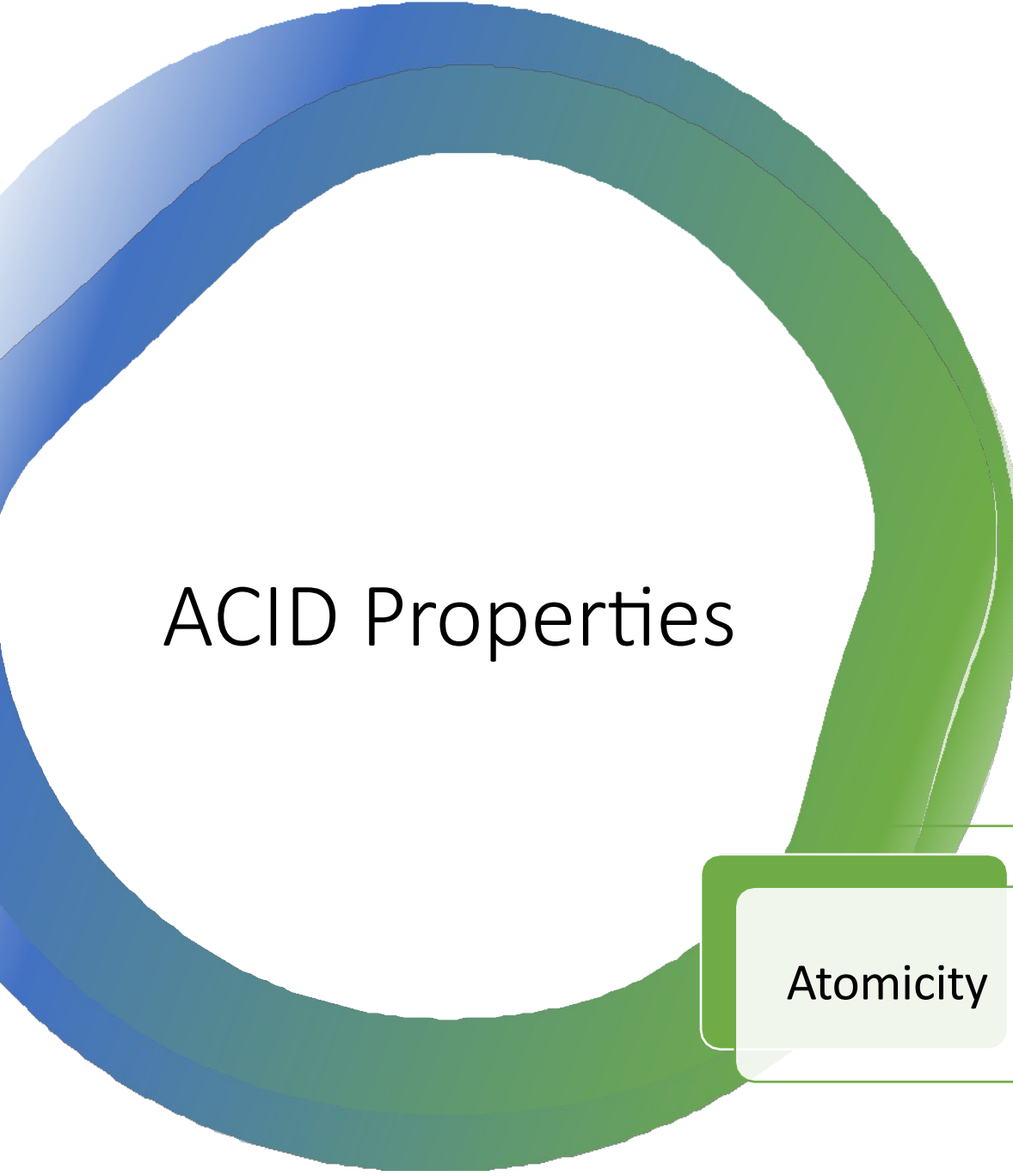
[ACID Properties in DBMS]



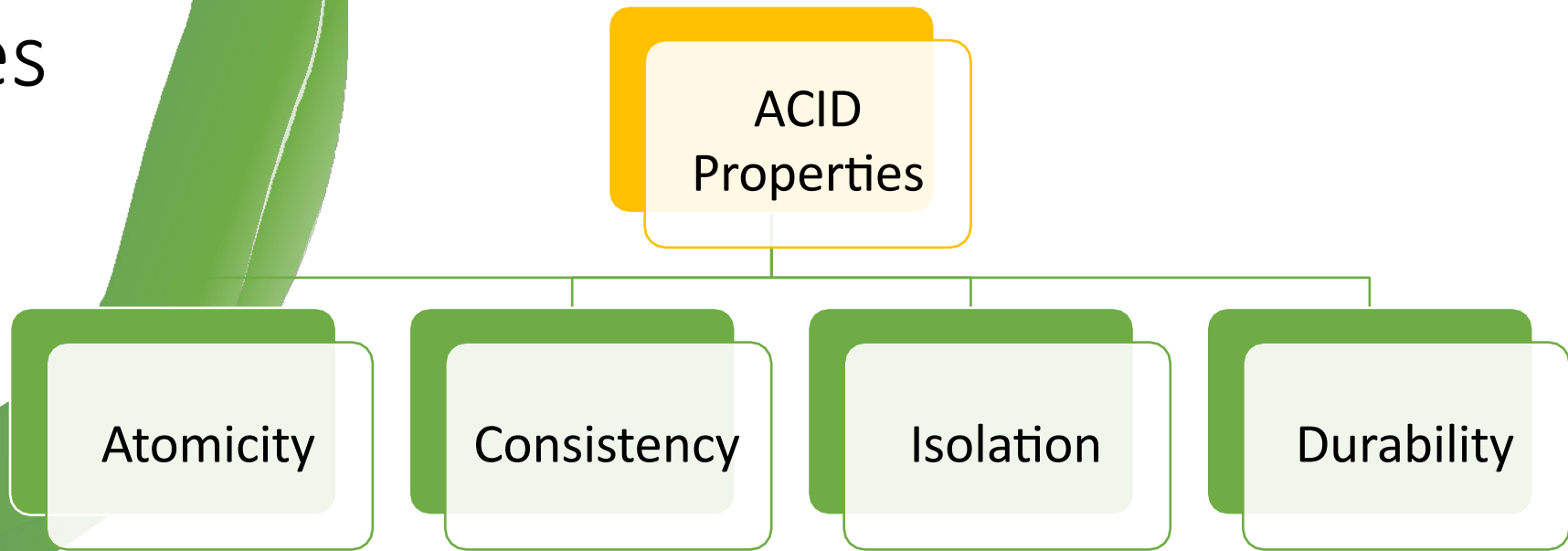
ACID Properties in DBMS

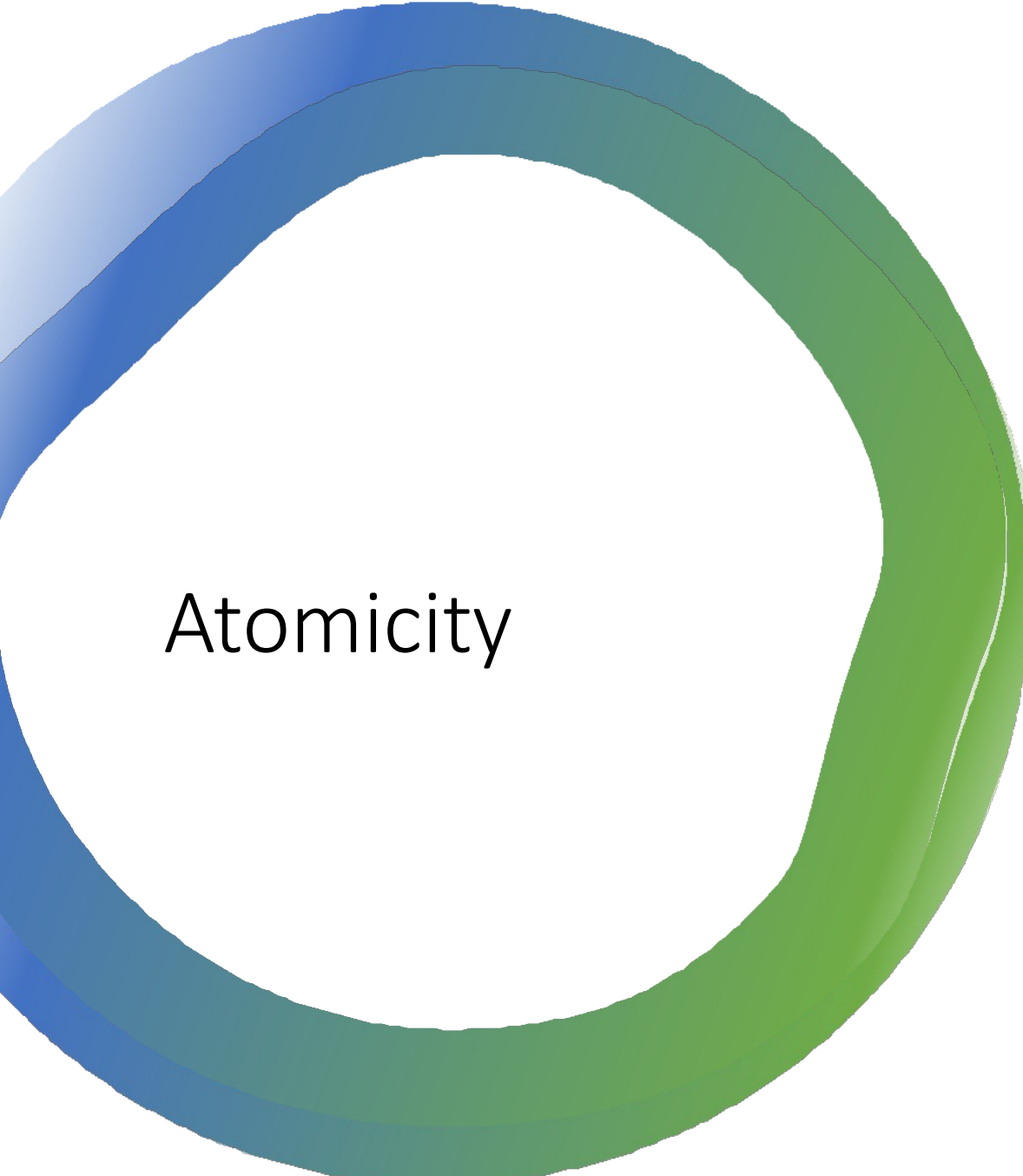
- DBMS is the management of data that should remain integrated when any changes are done in it. It is because if the integrity of the data is affected, whole data will get disturbed and corrupted. Therefore, to maintain the integrity of the data, there are four properties described in the database management system, which are known as the **ACID** properties.
- The ACID properties are meant for the transaction that goes through a different group of tasks, and there we come to see the role of the ACID properties.

ACID Properties



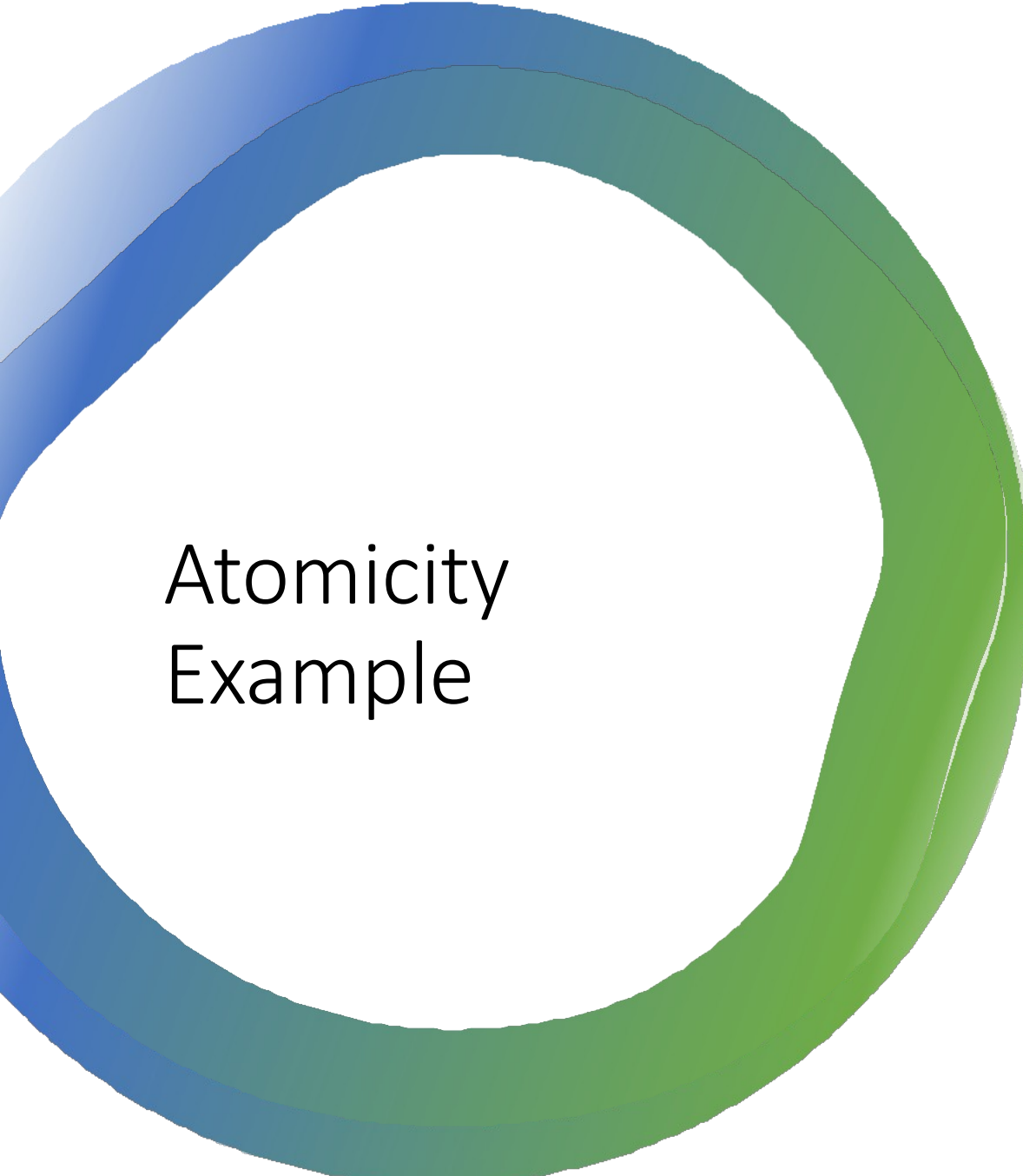
- The expansion of the term ACID defines for:





Atomicity

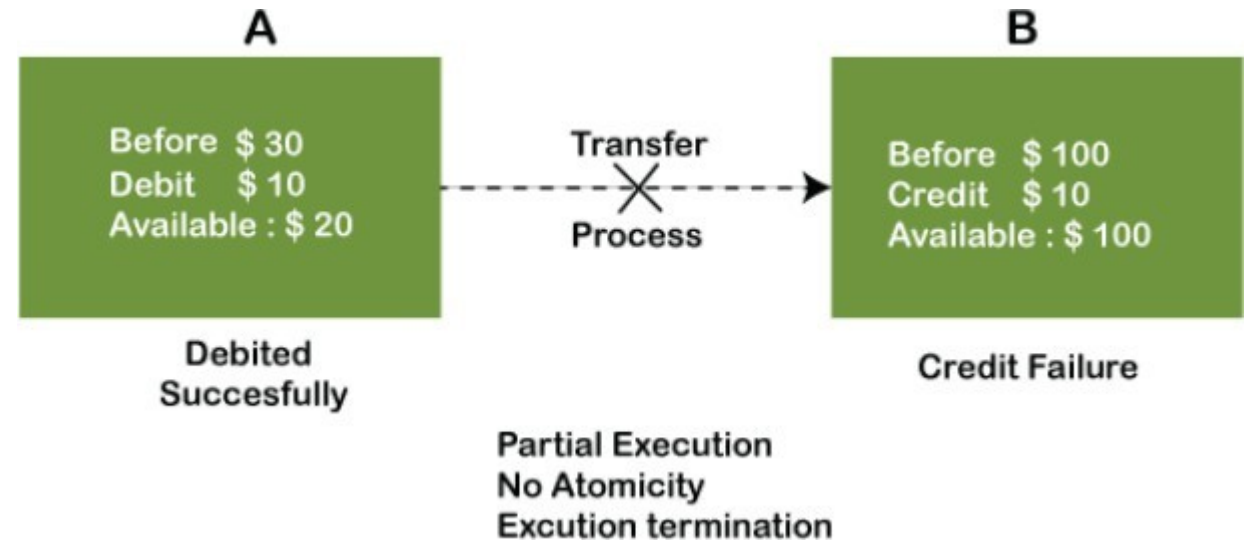
- The term atomicity defines that the data remains atomic.
- It means if any operation is performed on the data, either it should be performed or executed completely or should not be executed at all.
- It further means that the operation should not break in between or execute partially.
- In the case of executing operations on the transaction, the operation should be completely executed and not partially.



Atomicity Example

- If Kamil has account **A** having \$30 in his account from which he wishes to send \$10 to Saqib's account, which is **B**.
- In account **B**, a sum of \$100 is already present. When \$10 will be transferred to account **B**, the sum will become \$110.
- Now, there will be two operations that will take place.
 - One is the amount of \$10 that Kamil wants to transfer will be debited from his account **A**, and the same amount will get credited to account **B**, i.e., into Saqib's account.
- Now, what happens – the first operation of debit executes successfully, but the credit operation, however, fails.
- Thus, in Kamil's account **A**, the value becomes \$20, and to that of Saqib's account, it remains \$100 as it was previously present.

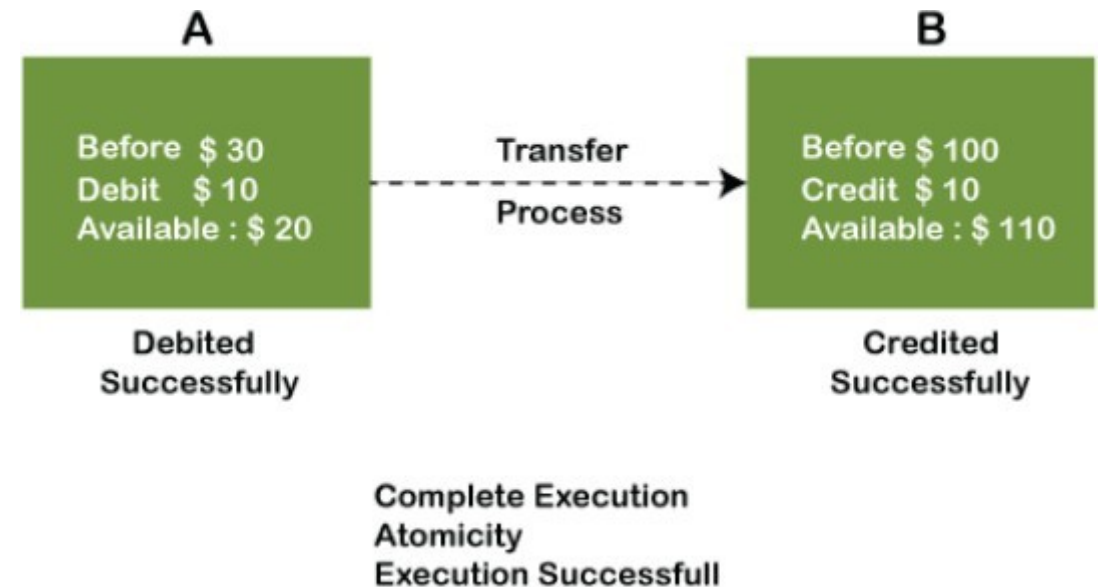
Atomicity Example



- In the above diagram, it can be seen that after crediting \$10, the amount is still \$100 in account B.
- So, it is not an ***atomic transaction***.

Atomicity Example

- The below image shows that both debit and credit operations are done successfully.
- Thus the ***transaction is atomic***.



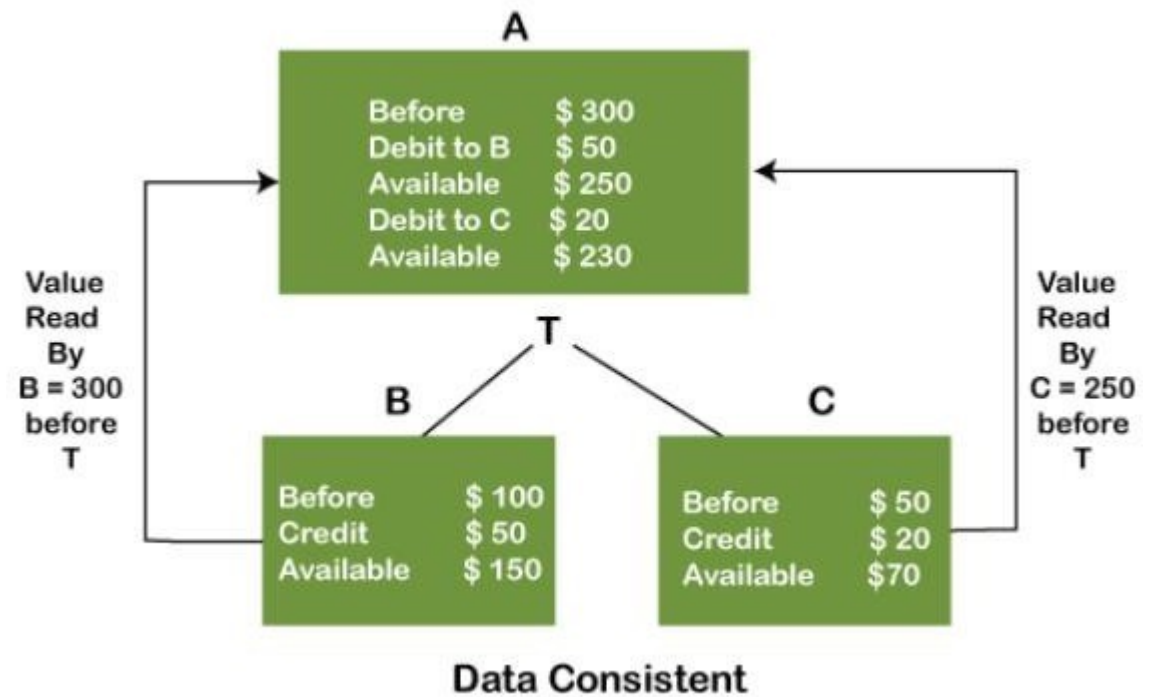
Thus, when the amount loses atomicity, then in the bank systems, this becomes a huge issue, and so the atomicity is the main focus in the bank systems.



Consistency

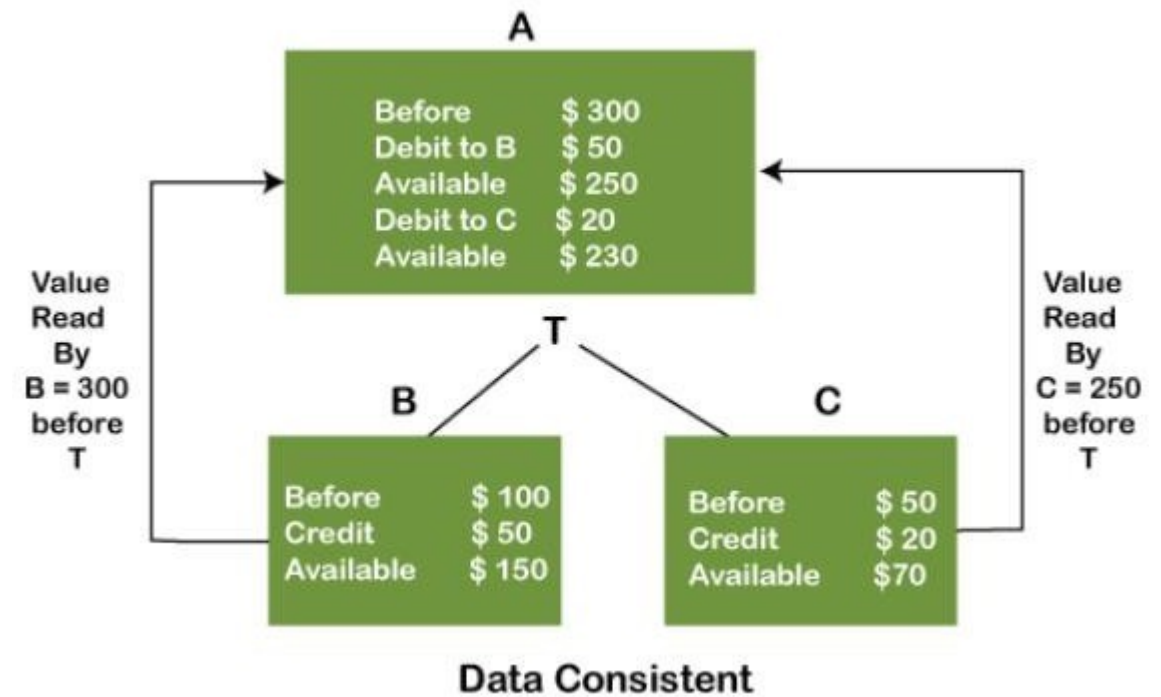
- The word **consistency** means that the value should remain preserved always.
- In DBMS, the integrity of the data should be maintained, which means if a change in the database is made, it should remain preserved always.
- In the case of transactions, the integrity of the data is very essential so that the database remains consistent before and after the transaction.
- The data should always be correct.

Consistency Example

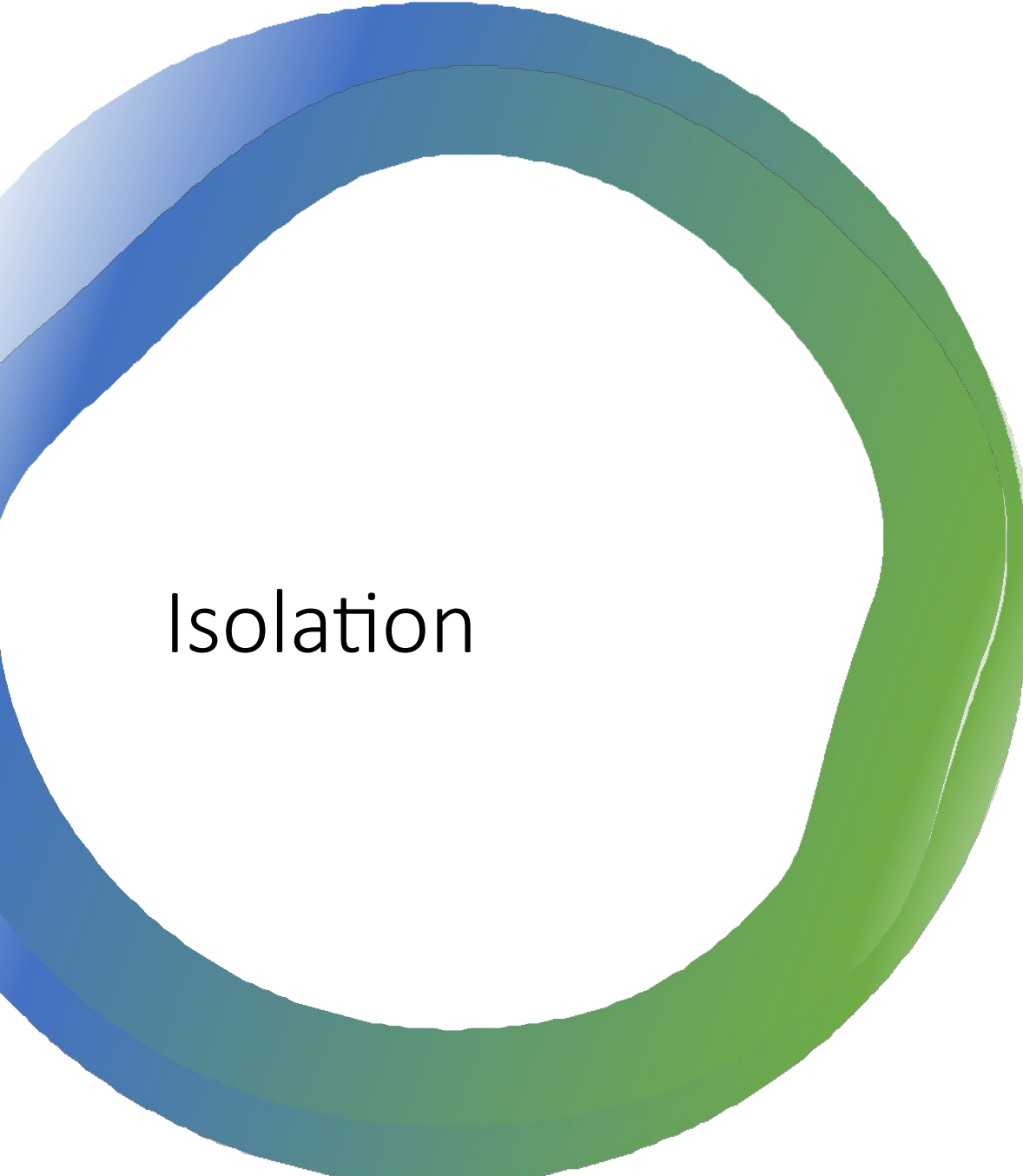


- There are three accounts, **A**, **B**, and **C**, where **A** is making a transaction **T** one by one to both **B** & **C**.
- There are two operations that take place, i.e., Debit and Credit.
- Account **A** firstly debits **\$50** to account **B**, and the amount in account **A** is read **\$300** by **B** before the transaction.
- After the successful transaction **T**, the available amount in **B** becomes **\$150**.

Consistency Example



- Now, **A** debits **\$20** to account **C**, and that time, the value read by **C** is **\$250** (that is correct as a debit of **\$50** has been successfully done to **B**).
- The debit and credit operation from account **A** to **C** has been done successfully. We can see that the transaction is done successfully, and the value is also read correctly.
- Thus, the data is **consistent**.
- In case the value read by **B** and **C** is **\$300**, which means that data is **inconsistent** because when the debit operation executes, it will not be consistent.

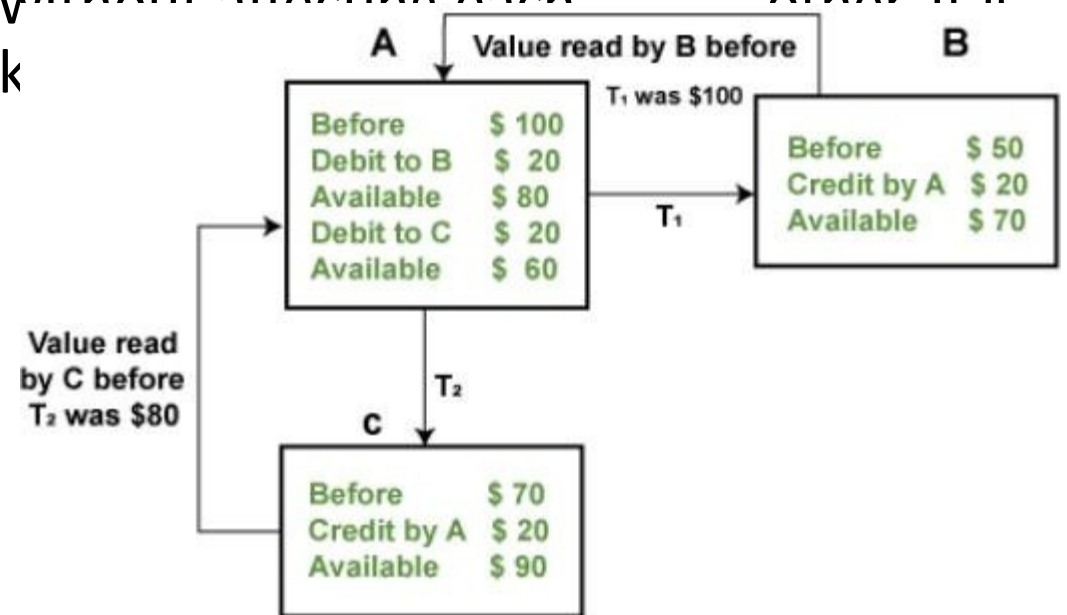


Isolation

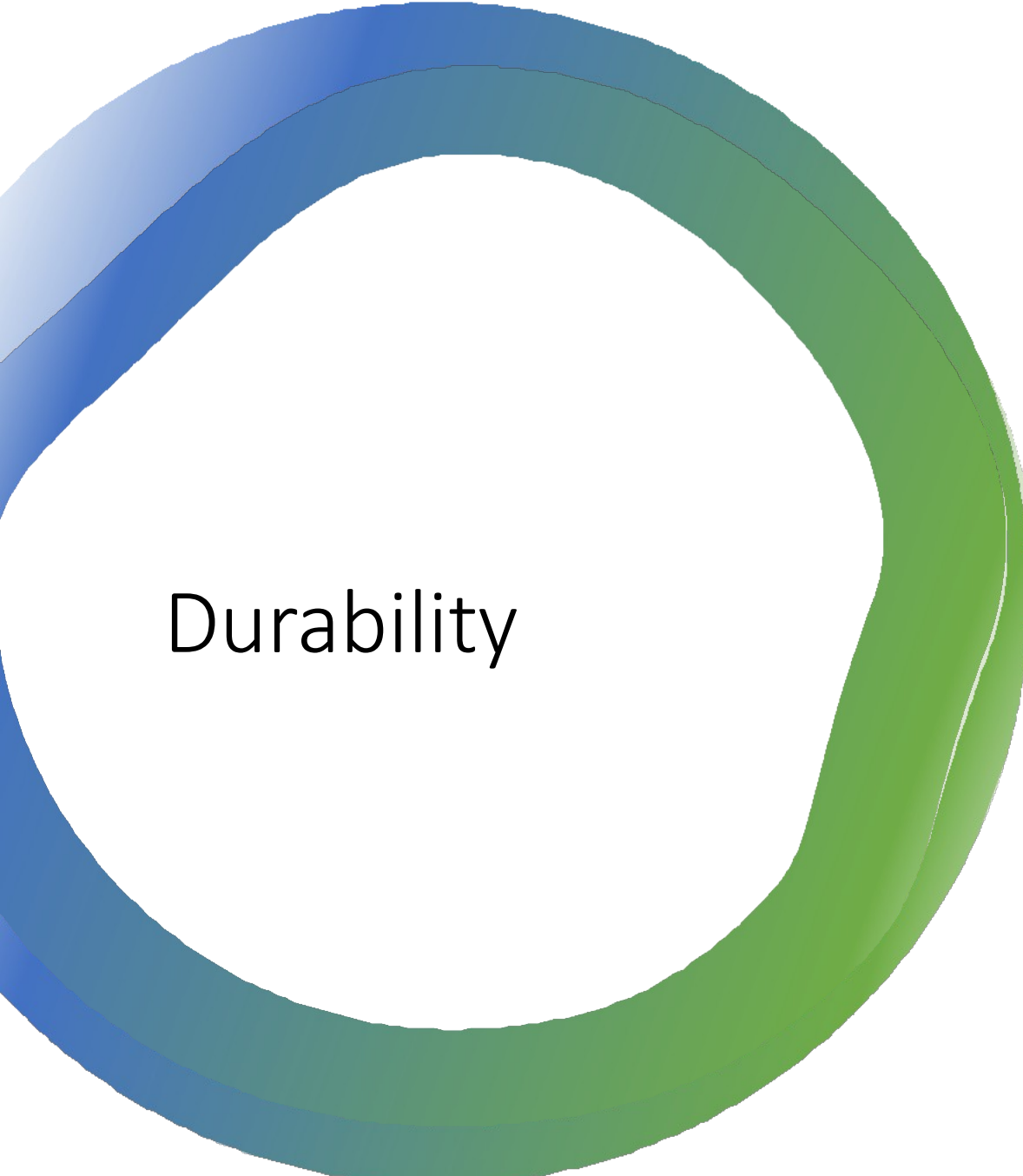
- The term '**isolation**' means *separation*.
- In DBMS, Isolation is the property of a database where no data should affect the other one and may occur concurrently.
- In short, the operation on one database should begin when the operation on the first database gets complete.
- It means if two operations are being performed on two different databases, they may not affect the value of one another.
- In the case of transactions, when two or more transactions occur simultaneously, the consistency should remain maintained.
- Any changes that occur in any particular transaction will not be seen by other transactions until the change is not committed in the memory.

Isolation Example

- If two operations are concurrently running on two different accounts, then the value of both accounts should not get affected. The value should remain persistent.
- As you can see in the below diagram, account **A** is making T_1 and T_2 transactions to account **B** and **C**, but both are executing independently without affecting each other. It is

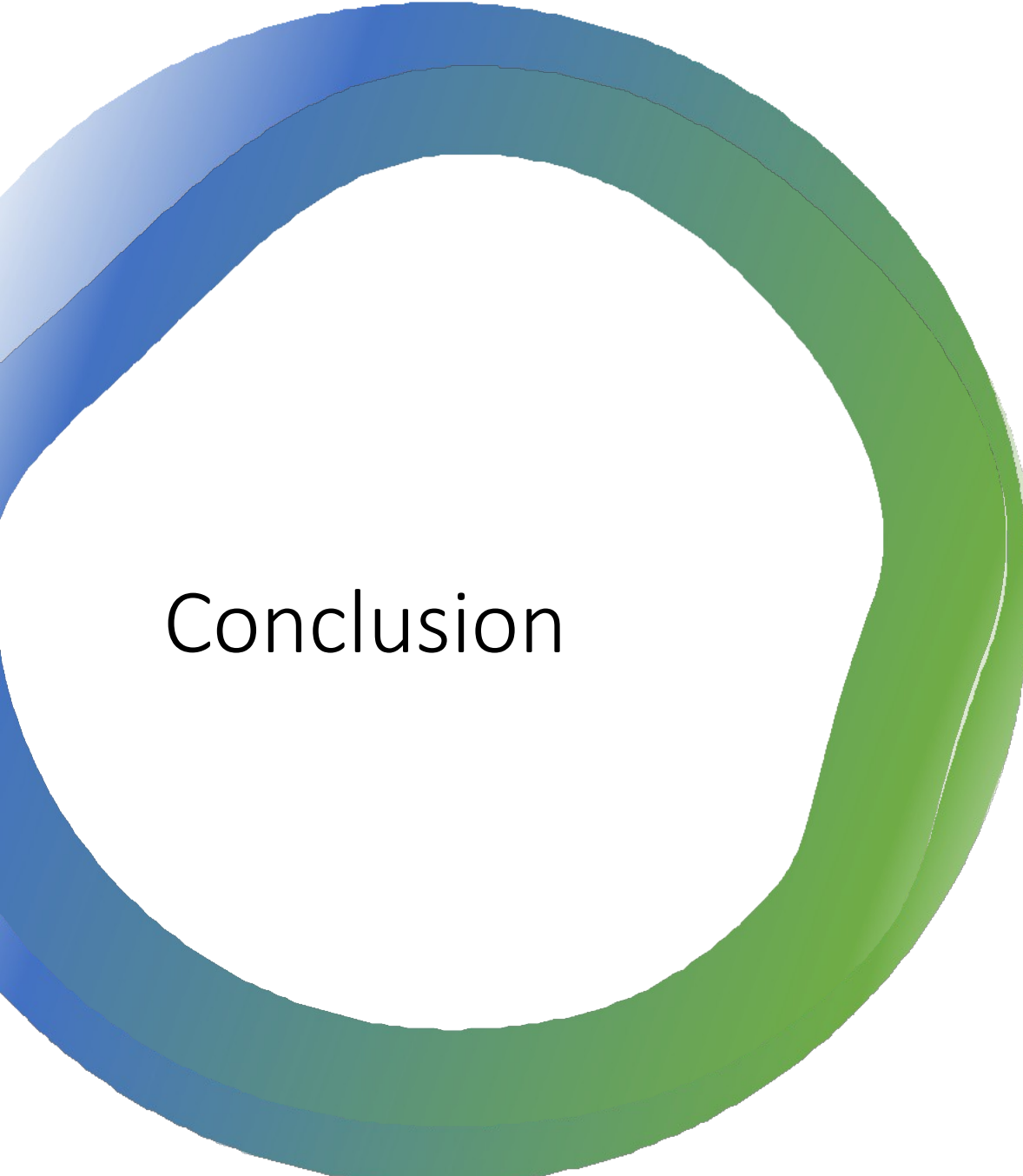


Isolation - Independent execution of T_1 & T_2 by A



Durability

- Durability ensures the ***permanency*** of something.
- In DBMS, the term durability ensures that the ***data after the successful execution of the operation becomes permanent*** in the database.
- The durability of the data should be so perfect that even if the system fails or leads to a crash, the database still survives.
- However, if gets lost, it becomes the responsibility of the recovery manager for ensuring the durability of the database. For committing the values, the COMMIT command must be used every time we make changes.



Conclusion

- Therefore, the ACID property of DBMS plays a vital role in maintaining the consistency and availability of data in the database.
- Thus, it was a precise introduction of ACID properties in DBMS. We have discussed these properties in the transaction section also.